

1. (12%) Case 1, Machine M with compiler X

Instruction class	CPI	Instruction count
A	3	1400
B	8	1800
C	5	1200
D	6	1100

Case 2, Machine M with compiler Y

Instruction class	CPI	Instruction count
A	3	1200
B	8	1700
C	5	1800
D	6	800

$$\begin{array}{r}
 2 \\
 23600 \\
 13600 \\
 9000 \\
 4800 \\
 \hline
 51000
 \end{array}$$

(1) (6%) Calculate the clock cycle of each case.

$$\begin{aligned}
 \text{Case 1: } & 3 \times 1400 + 8 \times 1800 + 5 \times 1200 + 6 \times 1100 \\
 & = 4200 + 14400 + 6000 + 6600 \\
 & = 31200
 \end{aligned}$$

$$\begin{aligned}
 \text{Case 2: } & 3 \times 1200 + 8 \times 1700 + 5 \times 1800 + 6 \times 800 \\
 & = 3600 + 13600 + 9000 + 4800 \\
 & = 31000
 \end{aligned}$$

(2) (2%) Following the previous question and answer which case has better effects?

ANS: Case 2

(3) (4%) How slow the clock rate should be to make the process has 0.5s difference?

$$(31200 - 31000) \div 0.5 = 400$$

A: 400 Hz

2. (12%) The following table shows the number of **instructions** for a program. Assuming that the **CPI** for the **R-format, load, store, and branch** instructions is **1, 4, 3** and **2, respectively**.

R-Format	Load	Store	Branch
40×10^9	20×10^9	10×10^9	30×10^9

- (1) (4%) Find the average CPI for the program.

$$\frac{(40 \times 10^9 \times 1 + 20 \times 10^9 \times 4 + 10 \times 10^9 \times 3 + 30 \times 10^9 \times 2)}{(40 + 20 + 10 + 30) \times 10^9}$$

$$= \frac{210}{100} = 2.1 \#$$

- (2) (4%) By how much we can improve the CPI of Load instructions if we want the program to run 1.25 times faster?

$$\frac{1}{1.25} = \frac{40 \times 10^9 \times 1 + 20 \times 10^9 \times x + 10 \times 10^9 \times 3 + 30 \times 10^9 \times 2}{40 \times 10^9 \times 1 + 20 \times 10^9 \times 4 + 10 \times 10^9 \times 3 + 30 \times 10^9 \times 2}$$

$$\frac{1}{1.25} = \frac{(40 + 20x + 30 + 60) \times 10^9}{(210) \times 10^9}$$

$$210 = (130 + 20x) 1.25$$

$$168 = 130 + 20x$$

$$38 = 20x$$

$$x = 1.9 \#$$

- (3) (4%) By how much is the total execution time of the program improved if the CPI of Load instructions is reduced by 25%, and the CPI of branch instructions is reduced by 40%?

【Improvement = (Old Time - New Time)/Old Time】

$$CPI_{old} = 2.1$$

$$CPI_{new} = \frac{40 \times 10^9 \times 1 + 20 \times 10^9 \times 3 + 10 \times 10^9 \times 3 + 30 \times 10^9 \times 1.2}{100 \times 10^9} = \frac{166}{100} = 1.66$$

$$\frac{2.1 - 1.66}{2.1} = 0.209$$

A: 20.9%

3. (10%) Translate the following MIPS code to C. Assume that the variables f, g, h, i, and j are assigned to registers \$s0, \$s1, \$s2, \$s3, and \$s4, respectively. Assume that the base address of the arrays A and B are in registers \$s6 and \$s7, respectively.

```

addi    $t0, $s6, 32    t0 = #A[8]
lw      $t0, 0($t0)     t0 = A[8]
sll     $t0, $t0, 4      t0 = A[8] * 16
sll     $t1, $s4, 2      t1 = j * 4
add     $t1, $t1, $s7    t1 = *B[j]
lw      $t1, 0($t1)     t1 = B[j]
add     $t0, $t0, $t1    t0 = A[8] * 16 + B[j]
add     $s3, $t0, $s2    i = A[8] * 16 + B[j] + h

```

ANS: $i = A[8] \times 16 + B[j] + h$

4. (10%) Provide the type and hexadecimal representation of following instruction:
【t0-8, s0-16, a0-4, v0-2, sub – func 34, sw – op 43】

(1) (5%) sub \$t5, \$t3, \$s2

R-Format

OP (6bits)	rs (5)	rt (5)	rd (5)	shamt (5)	function (6)
0	11 (t3)	18 (s2)	13 (t5)	0	34
0000 00	01 011 1 0010	0110 1	000 00	10 0010	
0x 0	1	7	2	6	8 2 2 #

(2) (5%) sw \$t3, 12(\$s1)

I-Format

OP (6)	rs (5)	rt (5)	immediate (16)
43	s1 (17)	t3 (11)	12
1010 11	10 001	0 1011	0000 0000 0000 1100
0x A	E	2 B	0 0 0 C #

5. (16%) Provide the type (4%), MIPS instruction (6%), and binary/Hexadecimal representation(6%) of instruction described by the following MIPS fields: (1) **op=43, rs=12, rt=22, imm= 240** , (2) **t2 = s1 + t0**

【t0=8, s0=16, a0=4, v0=2, add – func 32, sub – func 34, sw – op 43】

(1) OP (6) rs(5) rt(5) imm (16)

43 12 22 240

sw t4 s6 240

1010 11 01 100 1 0110 0000 0000 1111 0000

0x A D 9 6 0 0 F 0

I-Format : sw \$s6, 240(\$t4)

(2) add \$t2, \$s1, \$t0 R-Format

OP(6) rs(5) rt(5) rd(5) shamt(5) function(6)

0 17(s1) 8(t0) 10(t2) 0 32

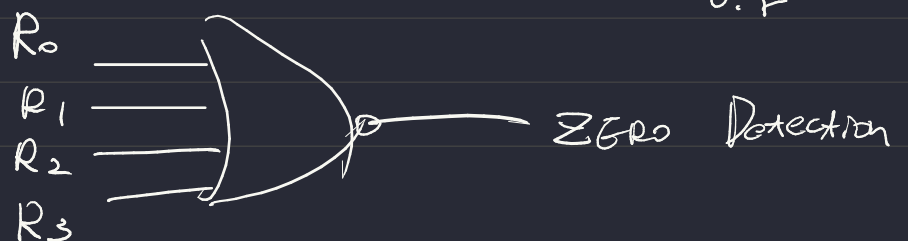
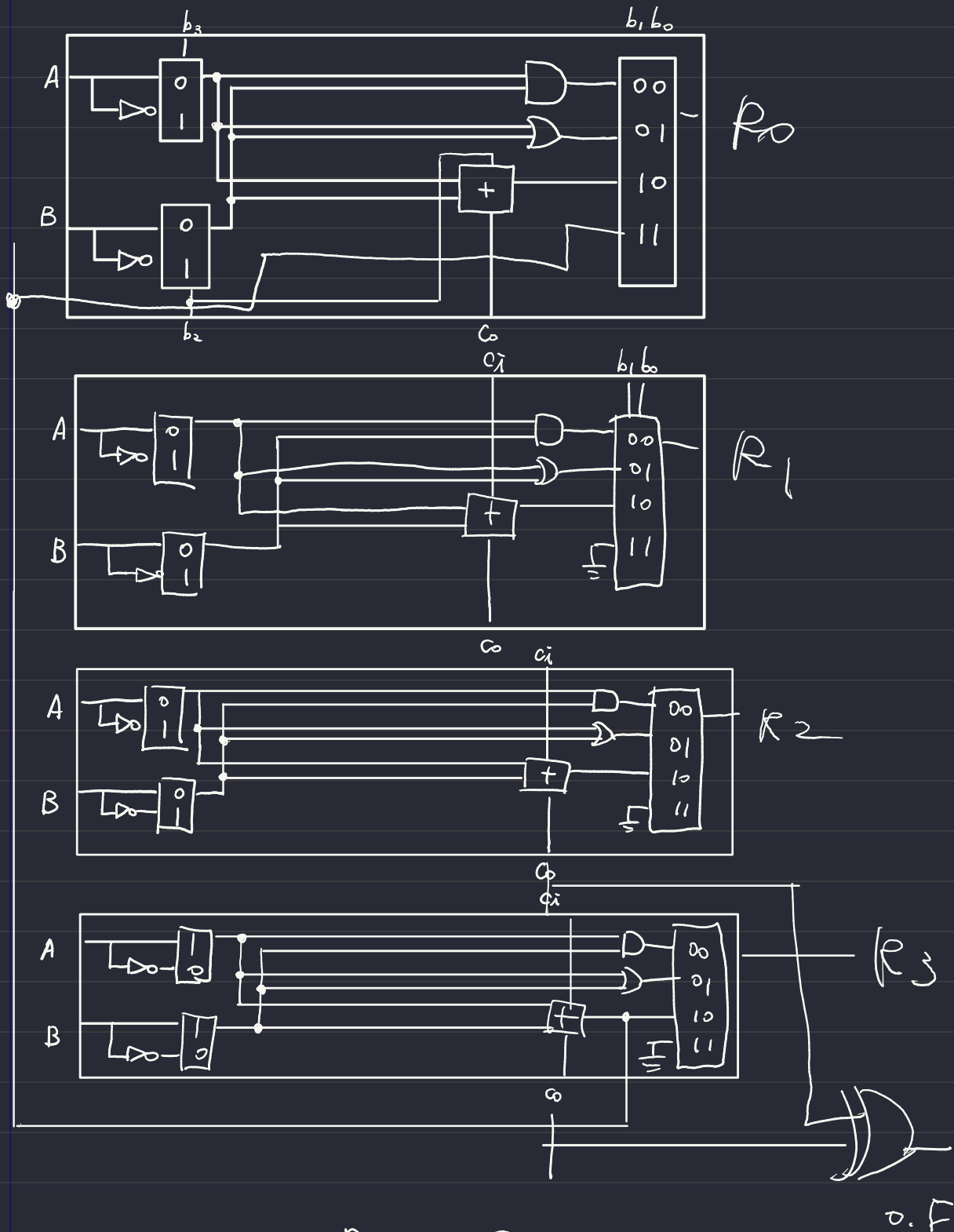
0000 00 10 001 0 1000 0101 0 000 00 100000

0x 0 2 2 8 5 0 2 0

7. (15%)

(1) Apply 1 bit ALU to draw 4 bit "set less than" logic unit. (5%)

(用 1 bit ALU 劃出 4 bit set less than 邏輯電路圖)



8. (15%) Carry lookahead is often used to speed up the addition operation in ALU. For a 4-bit addition with carry lookahead, assuming the two 4-bit inputs are $a_3a_2a_1a_0$ and $b_3b_2b_1b_0$, and the carry-in is c_0 ,

(1) (5%) Derive the recursive equations of carry-out c_{i+1} in terms of a_i and b_i and c_i , where $i = 0, 1, \dots, 3$.

$$c_{i+1} = a_i \cdot b_i + (a_i + b_i) \cdot c_i$$

(2) (5%) By defining the generate (g_i) and propagate (p_i) signals, express c_1, c_2, c_3 , and c_4 in terms of only g_i, p_i and c_0 .

$$c_1 = G_0 + P_0 \cdot c_0$$

$$c_2 = G_1 + P_1 \cdot (G_0 + P_0 \cdot c_0)$$

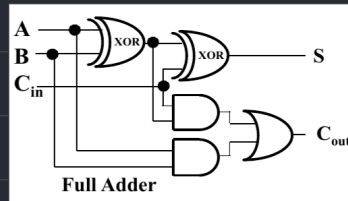
$$c_3 = G_2 + P_2 \cdot (G_1 + P_1 \cdot (G_0 + P_0 \cdot c_0))$$

$$c_4 = G_3 + P_3 \cdot (G_2 + P_2 \cdot (G_1 + P_1 \cdot (G_0 + P_0 \cdot c_0)))$$

(3) (5%) Estimate the speed up for this simple 4-bit carry lookahead adder over the 4-bit ripple carry adder (assuming each logic gate introduces T delay)

original :

$$2T \times 4 = 8T$$



Look ahead :

① G_i, P_i 要一層 T

② AND 的部份 一層 T

③ OR 的部份 一層 T

3T

$$8/3 = 2.67$$

答: 2.67倍